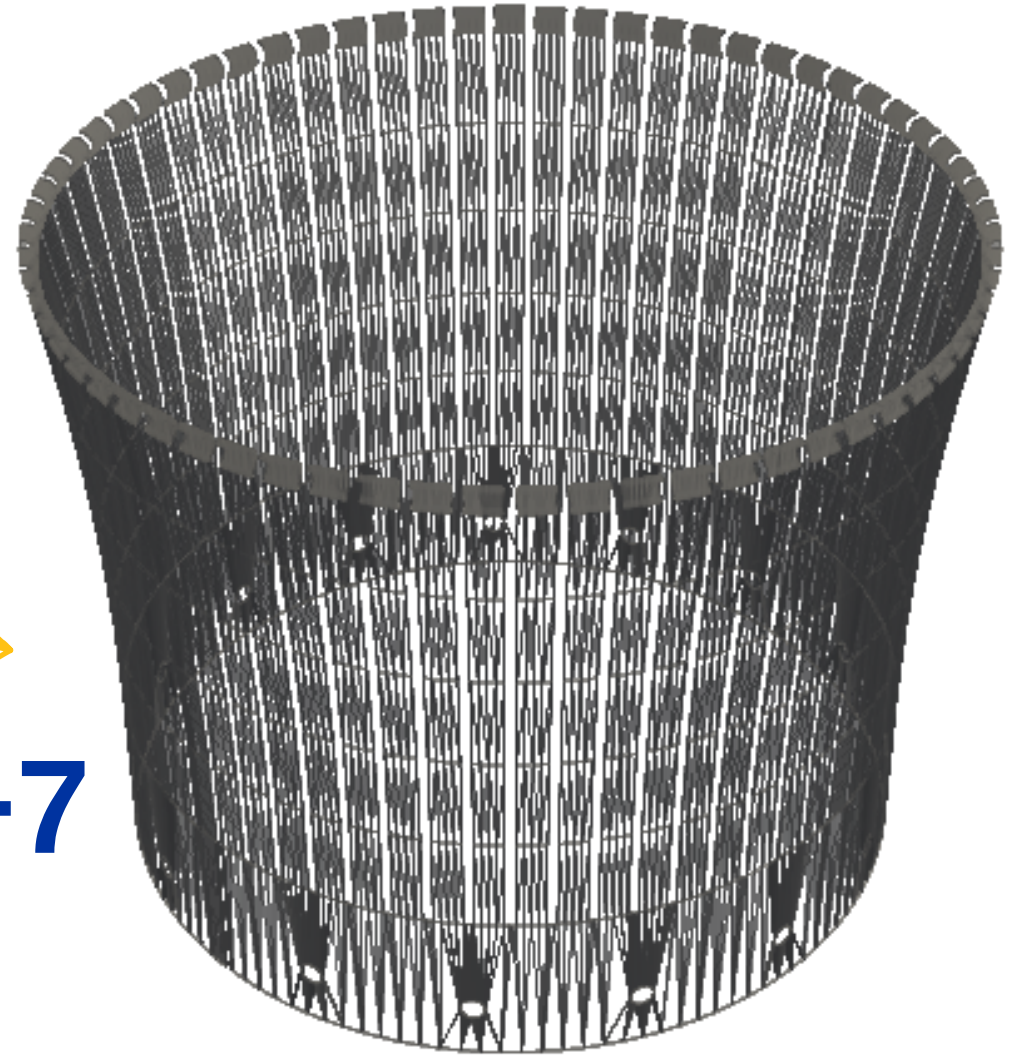
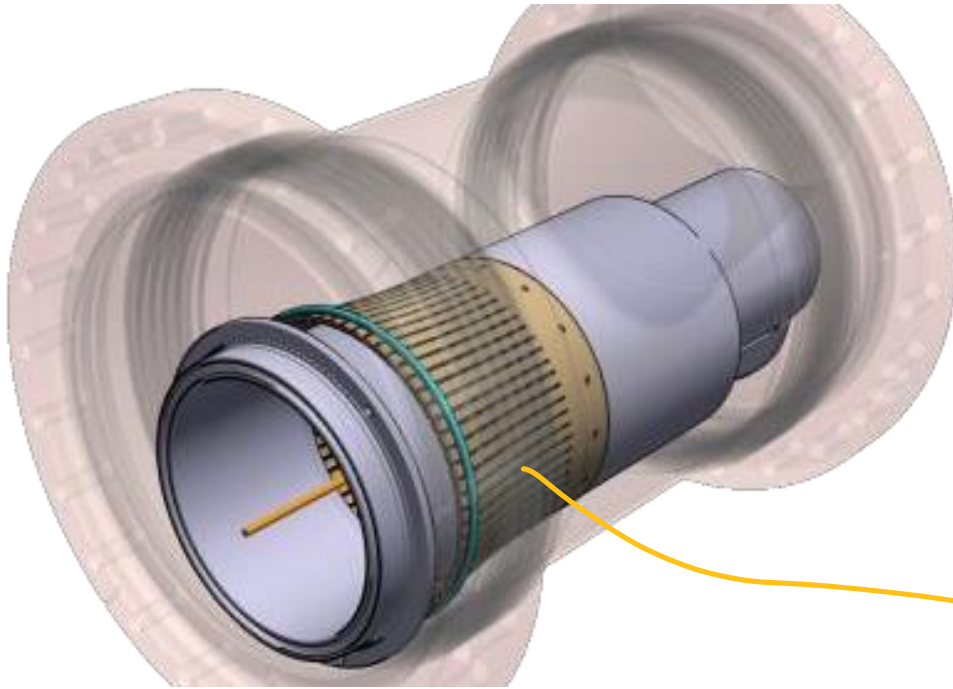




Project status week 6+7

Focus: Meshing of the Fingers

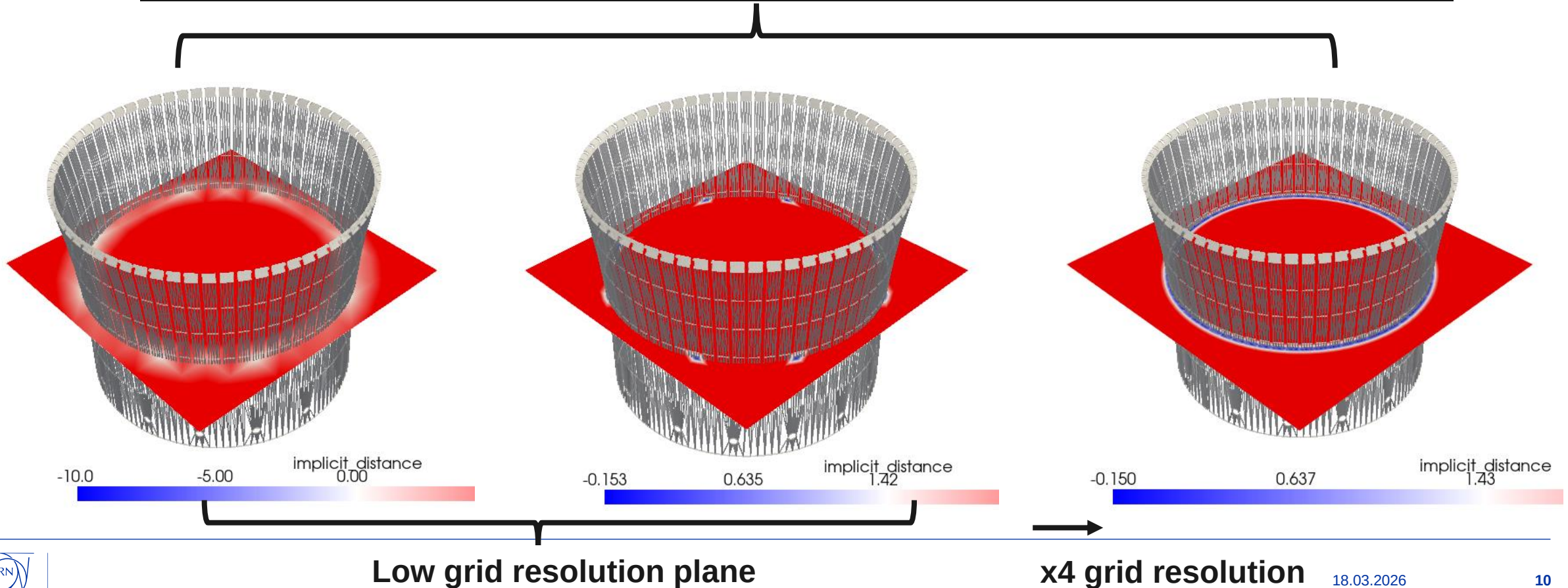
- studied the `compute_implicit_distances()` & point + cell mask retrievals
- Transformations on STL surface, including `subdivide()`
- Studied voxelize with and without `subdivide`
- Possibility of hybrid meshing-method: between voxelize & `implicit_distance`
- Studied grid resolution dependence & outlook discussion: snap points, smart meshing

Studying the `compute_implicit_distance` method for the Fingers I: The mechanisms behind the result

[-10,10] co
method. Di

```
plane = pv.Plane(center=(center_x, center_y, center_z), direction=(0, 0, 1),  
                 i_size=0.7*diagonal, j_size=0.7*diagonal,  
                 i_resolution=Nx-1, j_resolution=Nz-1)  
  
plane.compute_implicit_distance(surf_finger, inplace=True)
```

method
(signed)



Studying the compute_implicit_distance method for the Fingers II: *visualizing the mask of the previous slide's computed implicit distances (grid resolution 100x100x200) with varying cell mean distance mask thresholds:*

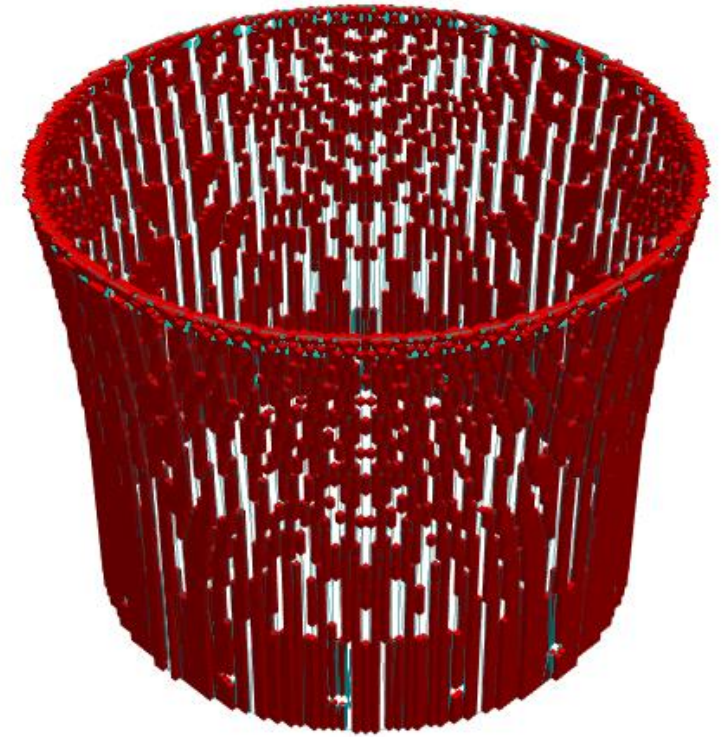
0.1



0.3



0.5

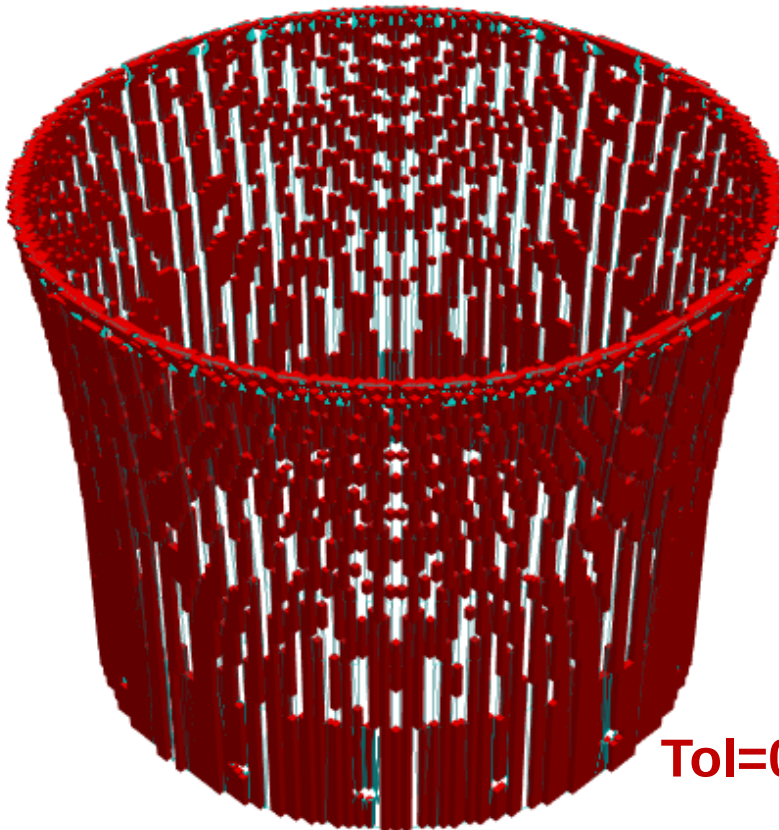


Studying the `compute_implicit_distance` method for the Fingers III: Which distance mask threshold to use?

How exactly is the cell mask constructed in Wakis
(Recap)

It has to **scale** with the
object & grid resolution

```
grid.c  
grid['
```



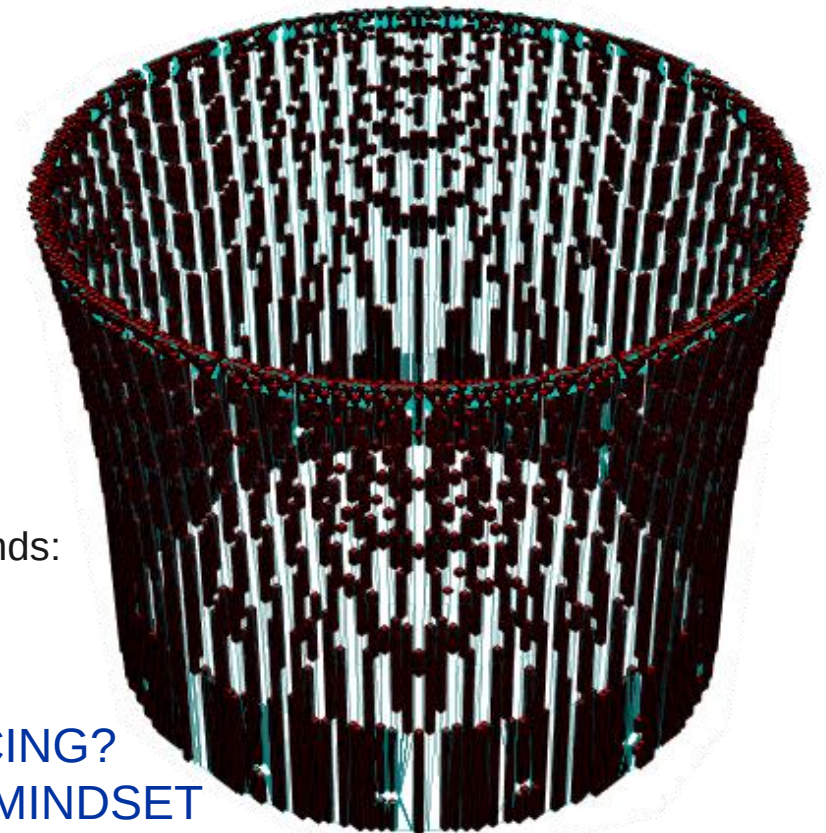
```
ce=True)  
implicit_distance'] <= tol
```

Tol = spacing

100x100x200 along surf.bounds:
Lx,Ly,Lz = 120.4, 120.4, 92
Spacing = 0.4625

Tol=0.5

BETTER WITH 0.x*SPACING?
GEOMETRIC-PHYSICS MINDSET

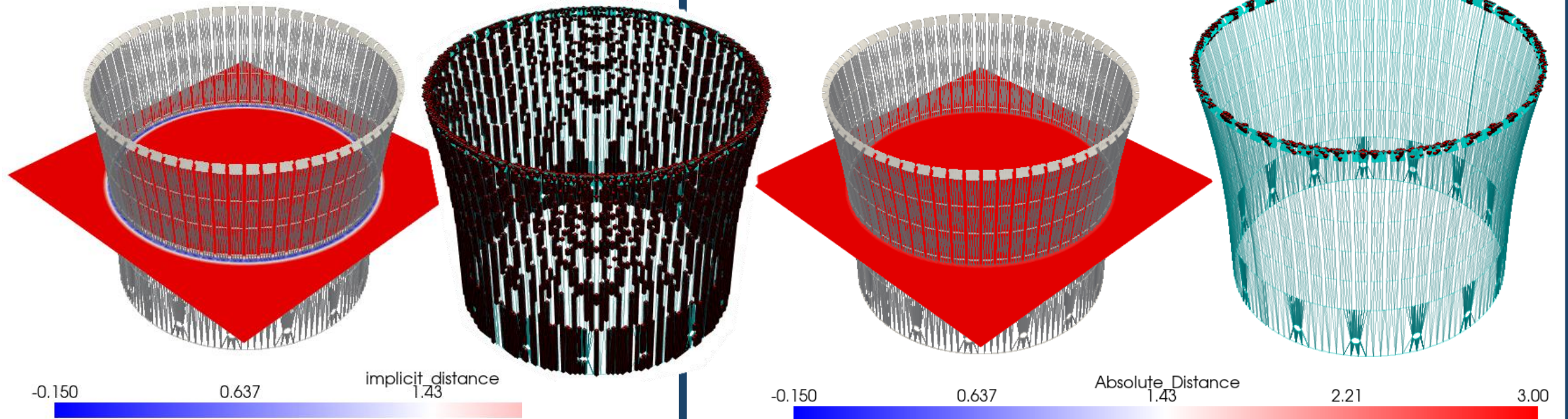


Studying the compute_implicit_distance method for the Fingers **V**: Quick comparison with absolute distance from **KDTree**

```
from scipy.spatial import KDTree
tree = KDTree(surf_finger.points)
d, idx = tree.query(plane.points)
plane["Absolute_Distance"] = d
```

Absolute distance calculation
to check if the **signed** normal projection
with individual triangle surfaces is
decreasing the voxelization quality

Extracted mask with
distance
threshold = spacing



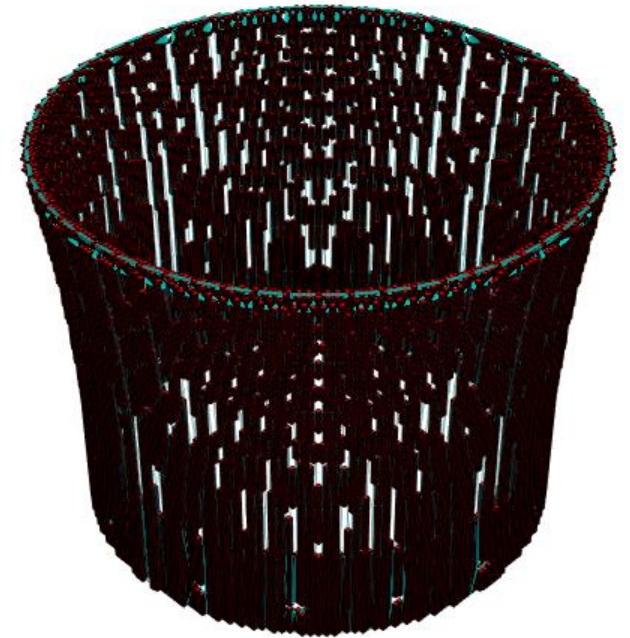
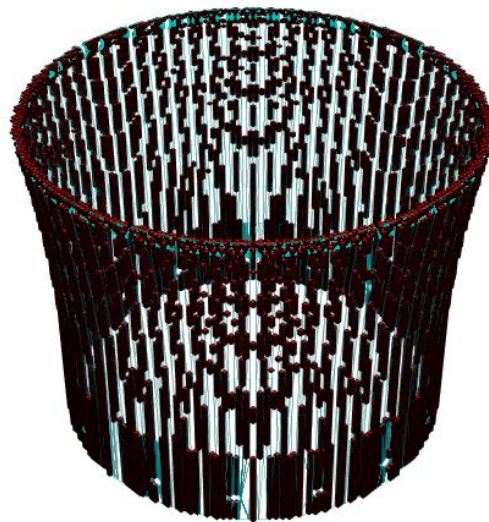
Same but for (signed) compute_implicit_distance

Studying the `compute_implicit_distance` method for the Fingers **VI**: *Transformations on the STL file*

`.clean()`: cleans STL surface by removing double triangles?.....

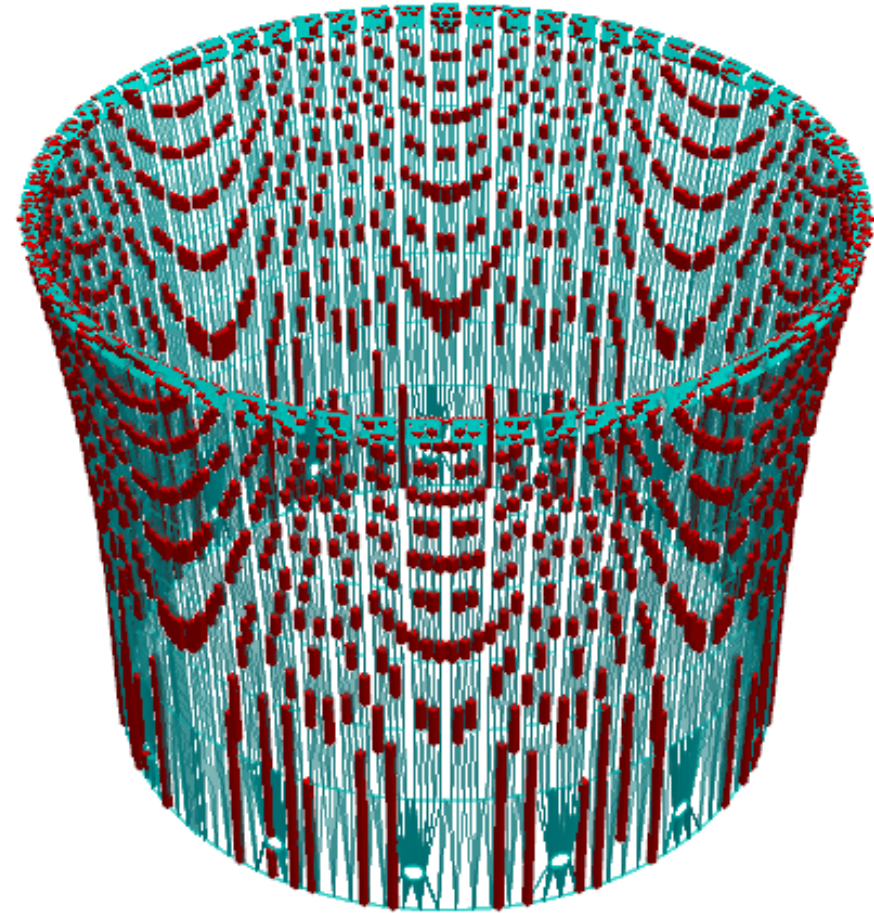
`.fill_holes(scalar)`: fills holes in the STL file of size up till the scalar

`.warp_by_scalar(factor)`: thickens the STL file by a factor



Studying the `voxelize_rectilinear()` method for the Fingers **II**:

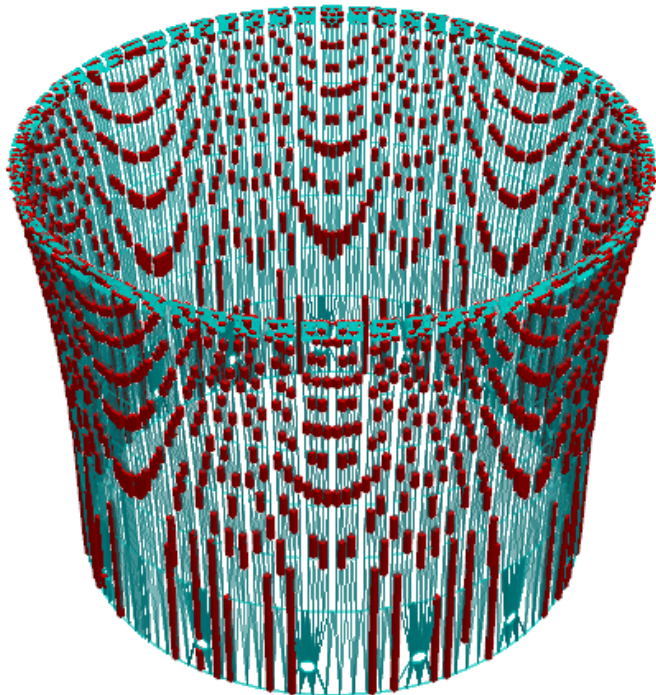
Voxelize without subdivide for grid resolution 100x100x200



Studying the `voxelize_rectilinear()` method for the Fingers

IV: with `.subdivide(nsub)`, grid res.100x100x200

Nsub=0



Nsub=1

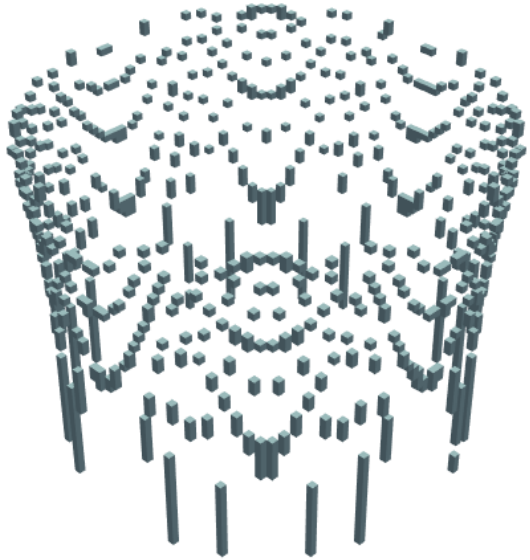


Nsub=2

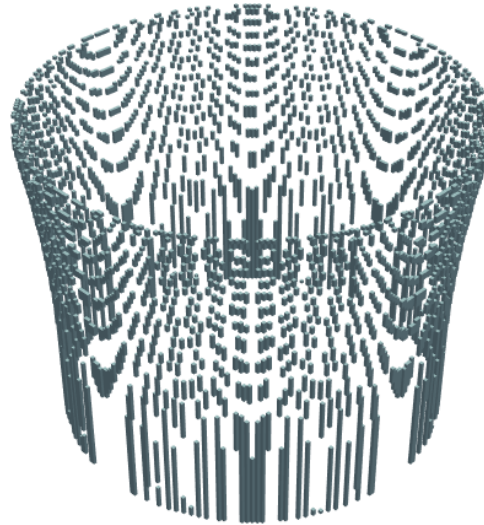


Studying the `voxelize_rectilinear()` method for the Fingers with `.subdivide(nsub=1)` scaling with grid resolution

25x25x50



50x50x50



100x100x200



200x200x400

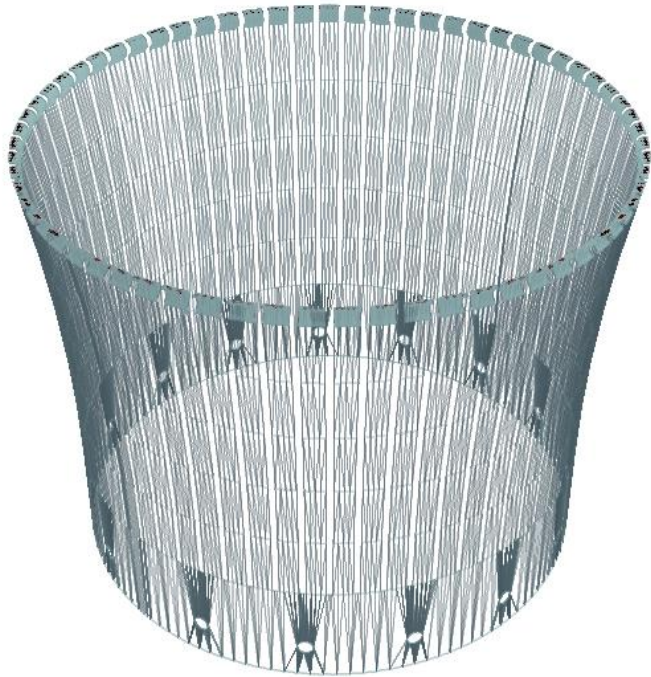


300x300x600



High Grid resolution compute_implicit_distance()

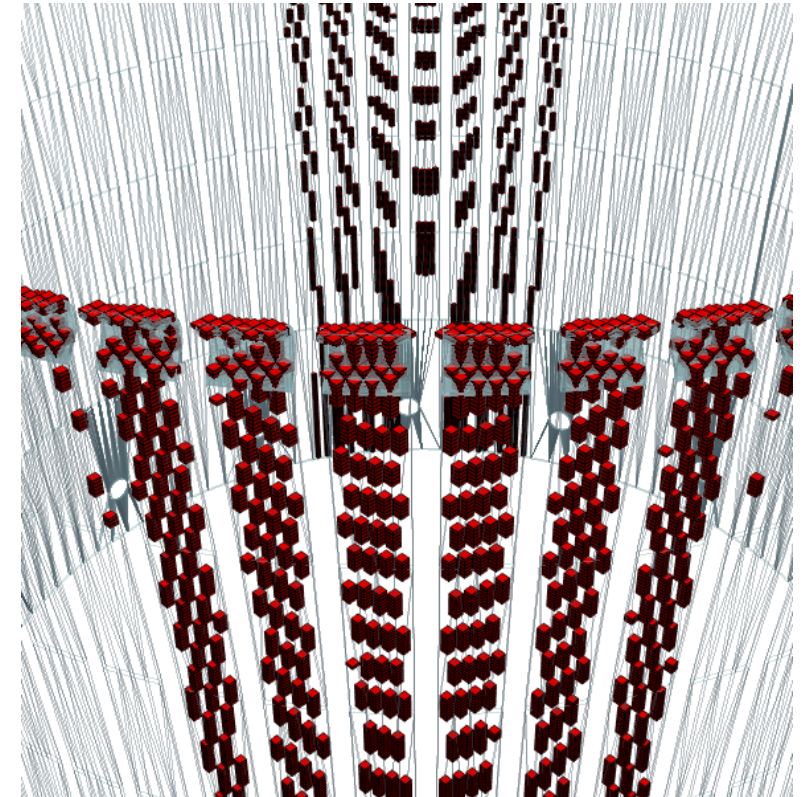
200 x 200 x 400, mask tol ≤ 0



200 x 200 x 400, mask tol $\leq 0.5 \times \text{spacing}$ (~ 0.23)



Zoom (of middle plot)

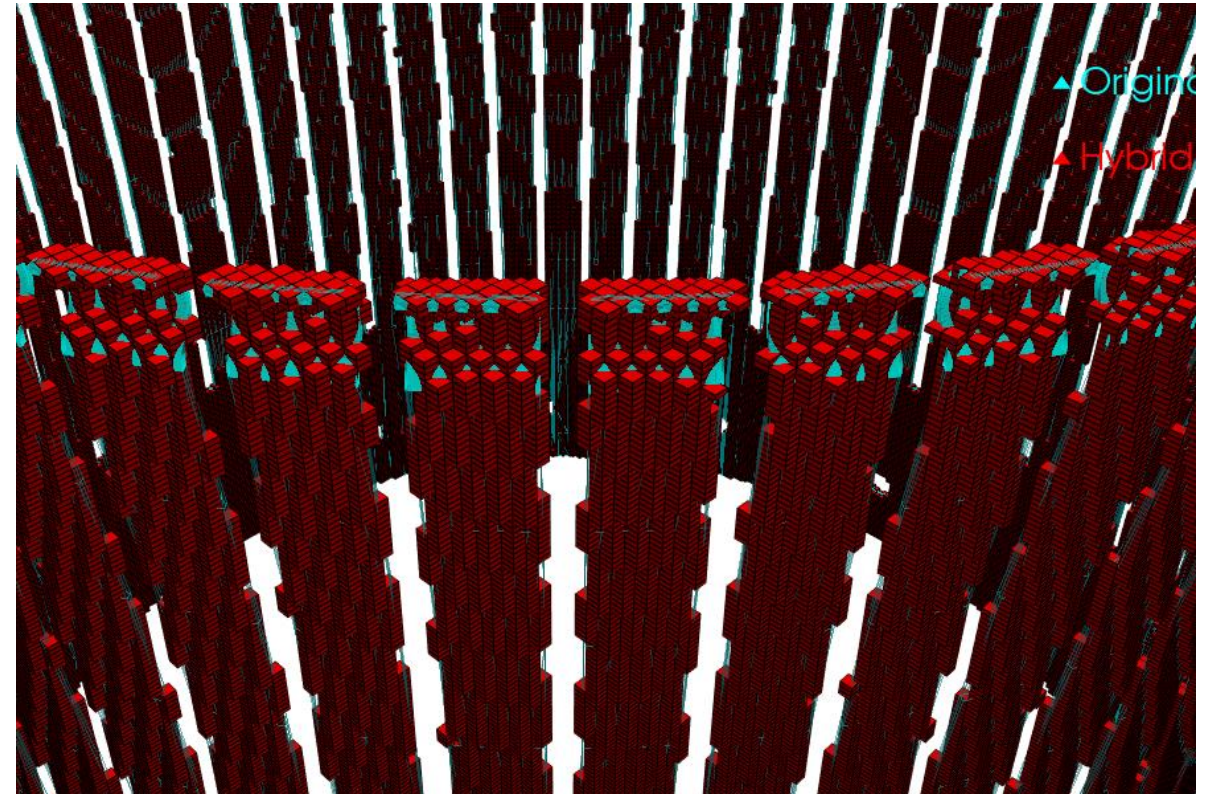


Studying hybrid approach I: between voxelize and implicit_distance

merging voxelize_rectilinear's robustness for thin surfaces with the signed distance field computed by compute_implicit_distance

Grid res. 200x200x400, mask
threshold=np.min(spacing), nsub=1

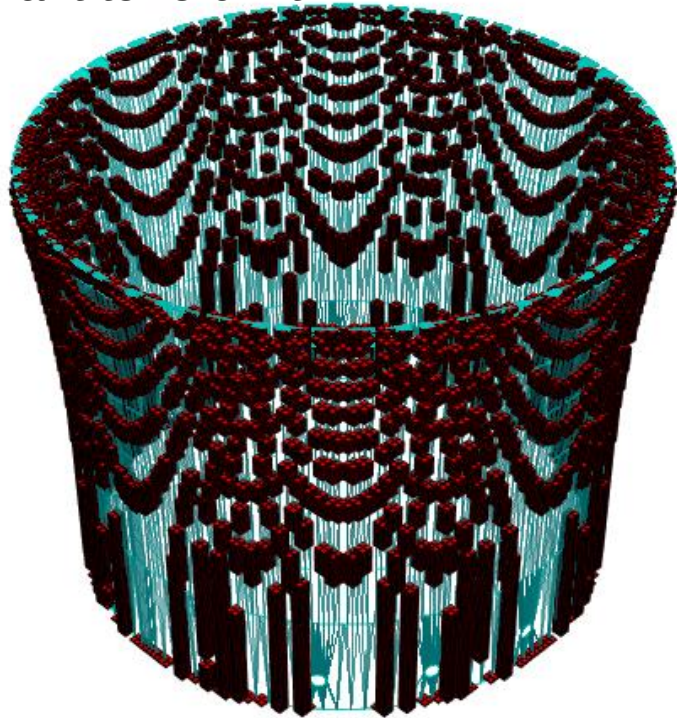
Zoom



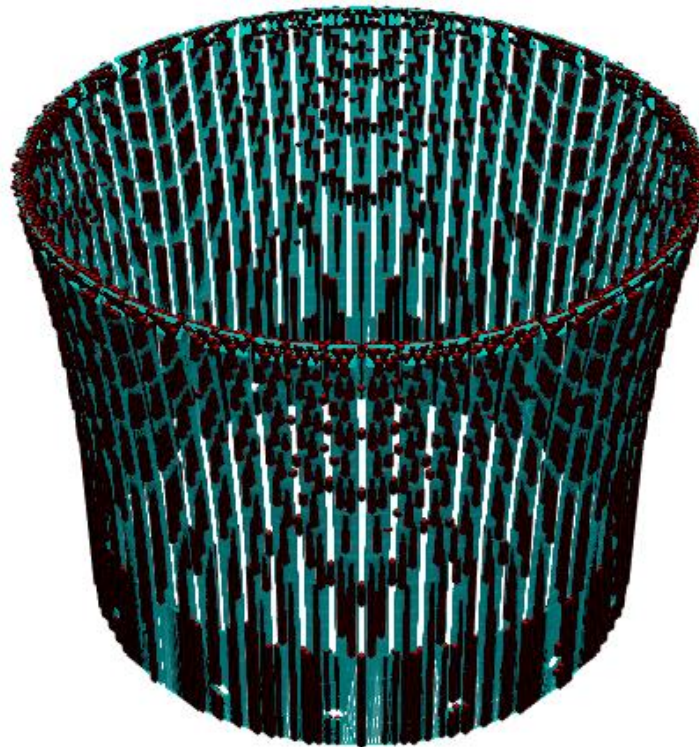
For Grid resolution 100x100x200 & implicit distance mask threshold of np.min spacing (~0.46)

Nsub=0

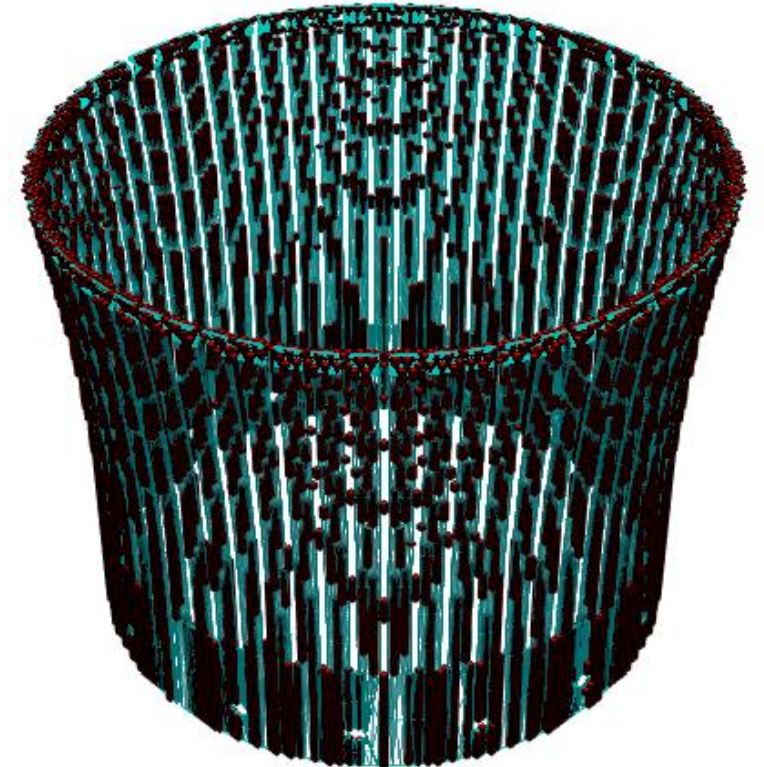
I think this one looks odd,
I want to rerun it



Nsub=1



Nsub=2



Gradient-based surface detection of implicit distance mask threshold=spacing mask

A bit bulky

Working on surface extraction from the voxelize mask

▲ Original STL
▲ Grid Interior

